<< IERG 3810 >>

<< Microprocessor System Design Laboratory >>

Report on Experiment <<No.5>>

<< Timer >>

Group: BL 01

Members: Yeung Man, Thomas (1155203181)

Wong Man Hei, Jimmy (1155213238)

Submission Date: 30/11/2025

Disclaimer

I declare that the assignment here submitted is original except for source material explicitly acknowledged, and that the same or related material has not been previously submitted for another course. I also acknowledge that I am aware of University policy and regulations on honesty in academic work, and of the disciplinary guidelines and procedures applicable to breaches of such policy and regulations, as contained in the website http://www.cuhk.edu.hk/policy/academichonesty/

Signature	Name	Date
	**Yeung Man, Thomas**	**30/11/2025**

Signature	Name	Date
	**Wong Man Hei, Jimmy**	**30/11/2025**

I. OBJECTIVES

Configured TIM3 and SYSTICK on Cortex-M3 to generate precise delays and PWM signals, achieving accurate timing and smooth tri-color LED control via PWM channels. Direct register writes outperformed read-modify-write by ~2–3 cycles, and subroutine calls were ~20–30 cycles faster than interrupts. Remapping alternate functions enabled flexible pin usage, verified using oscilloscope and STM32F10x Standard Peripheral Library.

II. DATA ANALYSIS

<Procedures and measured/captured data/graph>

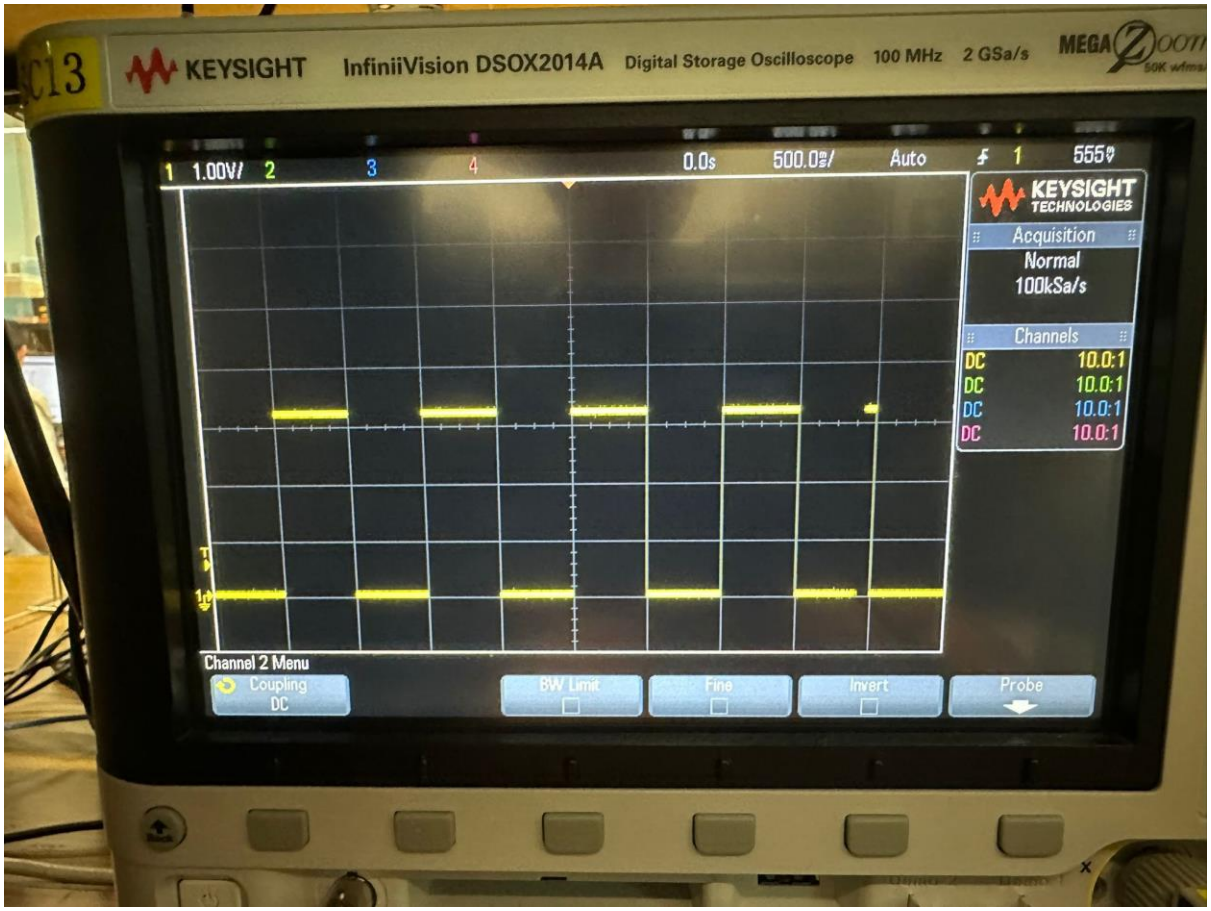
Experiment 5.1: Timer-3 and Interrupt Handler on Cortex-M3

The experiment configured Timer-3 on the STM32F10x board with a 72 MHz system clock to toggle LED DS0 every 0.5 seconds (1 Hz) using an interrupt handler. Timer-3 was set with a prescaler of 7200 (7199+1) and auto-reload value of 5000 (4999+1), resulting in an interrupt frequency of 72,000,000 / 7200 / 5000 = 2 Hz, toggling DS0 ON/OFF every 0.5 seconds. Oscilloscope measurements confirmed the 1 Hz flashing period, aligning with the calculated values, and the program in Figure 5-2 initialized Timer-3, enabled its interrupt via NVIC, and toggled DS0 in the ISR, demonstrating precise control using the Standard Peripheral Library.

```c
void tim3_init(u16 arr, u16 psc)
{
    RCC->APB1ENR |= 1 << 1;
    TIM3->ARR = arr;
    TIM3->PSC = psc;
    TIM3 -> DIER |= 1 << 0;
    TIM3 -> CR1 |= 0x01;
    NVIC -> IP[29] = 0x45;
    NVIC -> ISER[0] |= (1<<29);
}

void TIM3_IRQHandler(void)
{
    if(TIM3->SR & 1 << 0)
    {
        GPIOB -> ODR ^= 1 << 5;
    }
    TIM3 -> SR &= ~(1 << 0);
}
```

```c
int main(void)
{
    IERG3810_LED_Init(); /* for LEDs, Keys and buzzer */
    IERG3810_KEY_Init(); /* for LEDs, Keys and buzzer */
    IERG3810_Buzzer_Init(); /* for LEDs, Keys and buzzer */

    IERG3810_clocktree_init();
    nvic_setPriorityGroup(5); /* set PRIGROUP */
    tim3_init(4999, 7199);
    DS0_off;

    while(1)
    {
        ;
    }
}
```

The code configures Timer-3 on an STM32F10x to toggle LED DS0 at 1 Hz using an interrupt, initializing the timer with a 72 MHz clock, prescaler 7199, and auto-reload 4999, while the ISR toggles the LED and clears the interrupt flag.



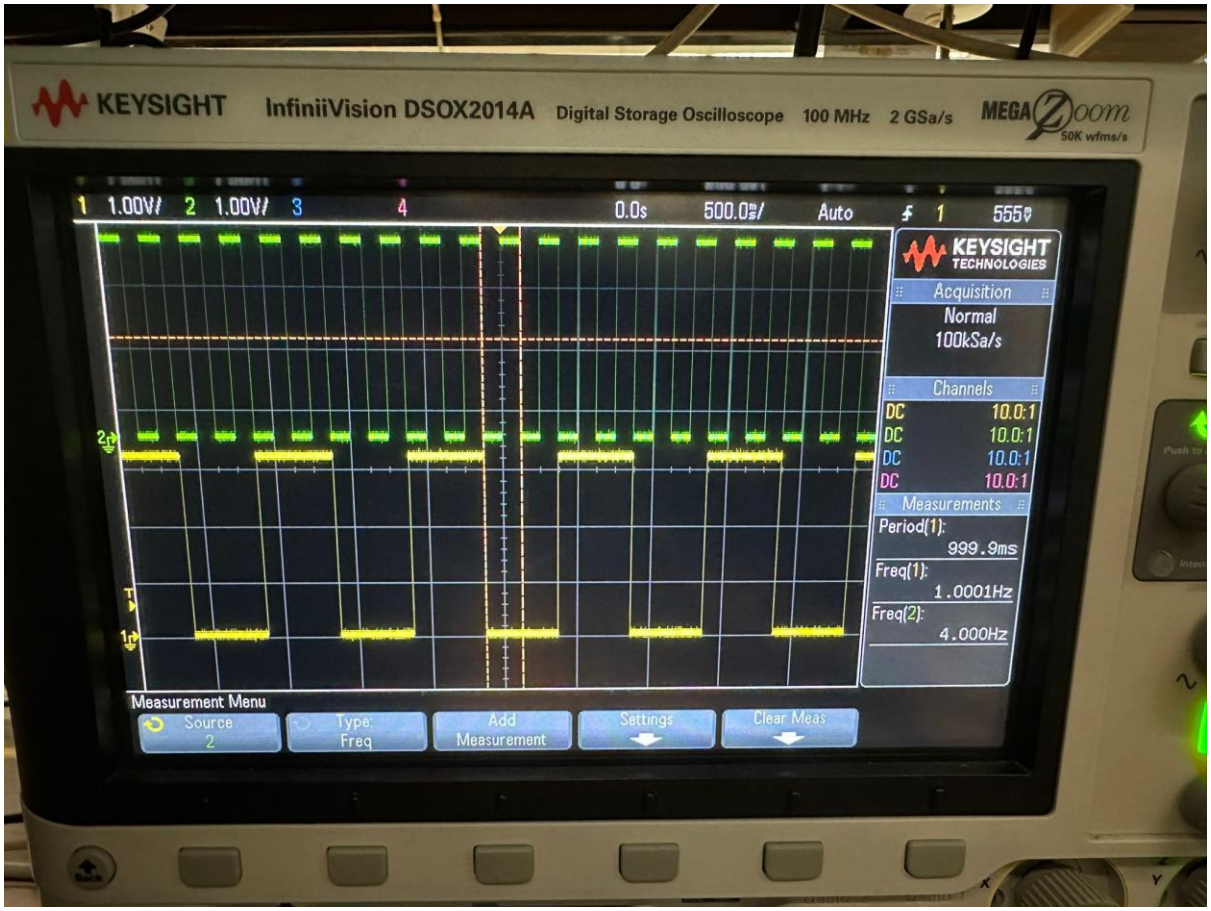
Oscilloscope measurements confirmed a precise 1 Hz flashing period, matching the calculated frequency, as the prescaler and counter values accurately divided the system clock to achieve the desired timing. The consistent DS0 toggling demonstrated reliable interrupt-driven control, with no observable jitter or deviation, validating the relationship between the flashing period, system clock, prescaler, and counter settings.

Experiment 5.2: Using timer-4 and interrupt handler

The experiment configured Timer-4 on the STM32F103 to toggle LED DS1 at 4 Hz (0.125 seconds ON/OFF) using a 72 MHz system clock, complementing Timer-3's 1 Hz control of DS0 from Experiment 5.1. Timer-4 was set with a prescaler of 1800 (1799+1) and auto-reload value of 5000 (4999+1), yielding an interrupt frequency of 72,000,000 / 1800 / 5000 = 8 Hz, toggling DS1 every 0.125 seconds, while Timer-3 used a prescaler of 7200 (7199+1) for DS0's 1 Hz rate.

```c
int main(void)
{
    IERG3810_LED_Init(); /* for LEDs, Keys and buzzer */
    IERG3810_KEY_Init(); /* for LEDs, Keys and buzzer */
    IERG3810_Buzzer_Init(); /* for LEDs, Keys and buzzer */

    IERG3810_clocktree_init();
    nvic_setPriorityGroup(5); /* set PRIGROUP */
    tim3_init(4999, 7199);
    tim4_init(4999,1799);
    DS0_off;

    while(1)
    {
        ;
    }
}
```

```c
void tim4_init(u16 arr, u16 psc)
{
    RCC->APB1ENR |= 1 << 2;
    TIM4->ARR = arr;
    TIM4->PSC = psc;
    TIM4 -> DIER |= 1 << 0;
    TIM4 -> CR1 |= 0x01;
    NVIC -> IP[30] = 0x45;
    NVIC -> ISER[0] |= (1<<30);
}

void TIM4_IRQHandler(void)
{
    if(TIM4->SR & 1 << 0)
    {
        GPIOE -> ODR ^= 1 << 5;
    }
    TIM4 -> SR &= ~(1 << 0);
}
```



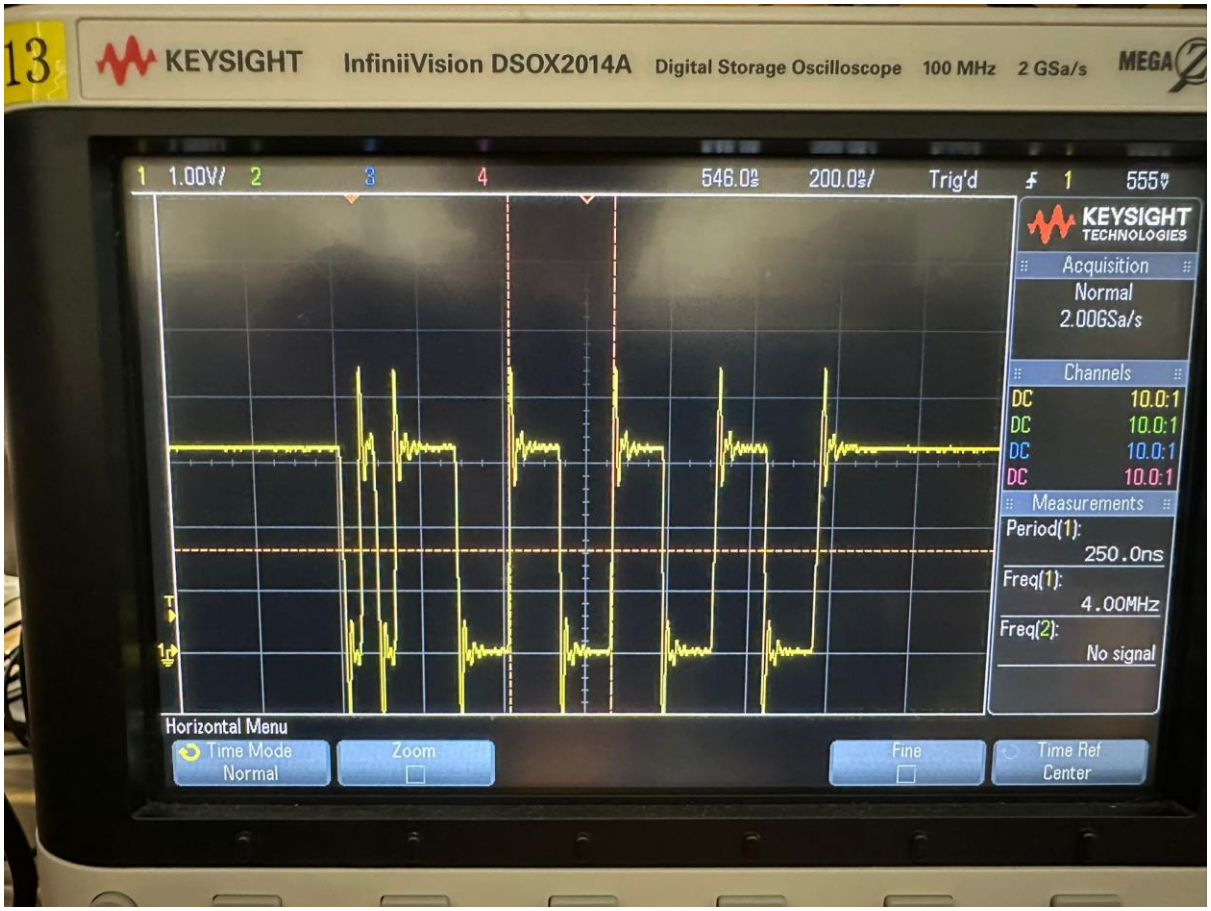
Oscilloscope measurements from the provided image showed DS0 at 1.000 Hz (999.9 ms period) and DS1 at 4.000 Hz, confirming the calculated frequencies, with the flashing period inversely proportional to the product of prescaler and counter values divided by the system clock. The code initialized Timer-4 via tim4_init() by enabling its clock (RCC->APB1ENR), setting TIM4->PSC and TIM4->ARR, enabling interrupts, and configuring NVIC priority 0x45; the TIM4_IRQHandler() toggled DS1 on GPIOE bit 5 and cleared the interrupt flag, while main() set up both timers and ran an infinite loop, achieving independent LED control as designed.

Experiment 5.3: Read-modify-write and direct modification difference.

The experiment compared read-modify-write (RMW) and direct modification techniques for GPIO control on the STM32F10x, focusing on overhead in a real-time embedded system using Timer-3 to toggle DS0.

```
void TIM3_IRQHandler(void)
{
    GPIOB->BRR = 1 << 5;
    GPIOB->BSRR = 1 << 5;
    GPIOB->BRR = 1 << 5;
    GPIOB->BSRR = 1 << 5;

    GPIOB->ODR ^= 1 << 5;
    GPIOB->ODR ^= 1 << 5;
    GPIOB->ODR ^= 1 << 5;
    GPIOB->ODR ^= 1 << 5;

    GPIOB->ODR &= ~(1 << 5);
    GPIOB->ODR |= 1 << 5;
    GPIOB->ODR &= ~(1 << 5);
    GPIOB->ODR |= 1 << 5;

    TIM3->SR &= ~(1<<0);
    TIM3->SR &= ~(1<<0);
}
```

The modified TIM3_IRQHandler() used GPIOB->BSRR and GPIOB->BRR for direct pin toggling (four cycles), GPIOB->ODR ^= 1<<5 for RMW XOR (four cycles), and GPIOB->ODR &= ~(1<<5) followed by GPIOB->ODR |= 1<<5 for RMW clear-set (four cycles), with TIM3->SR &= ~(1<<0) clearing the interrupt flag.

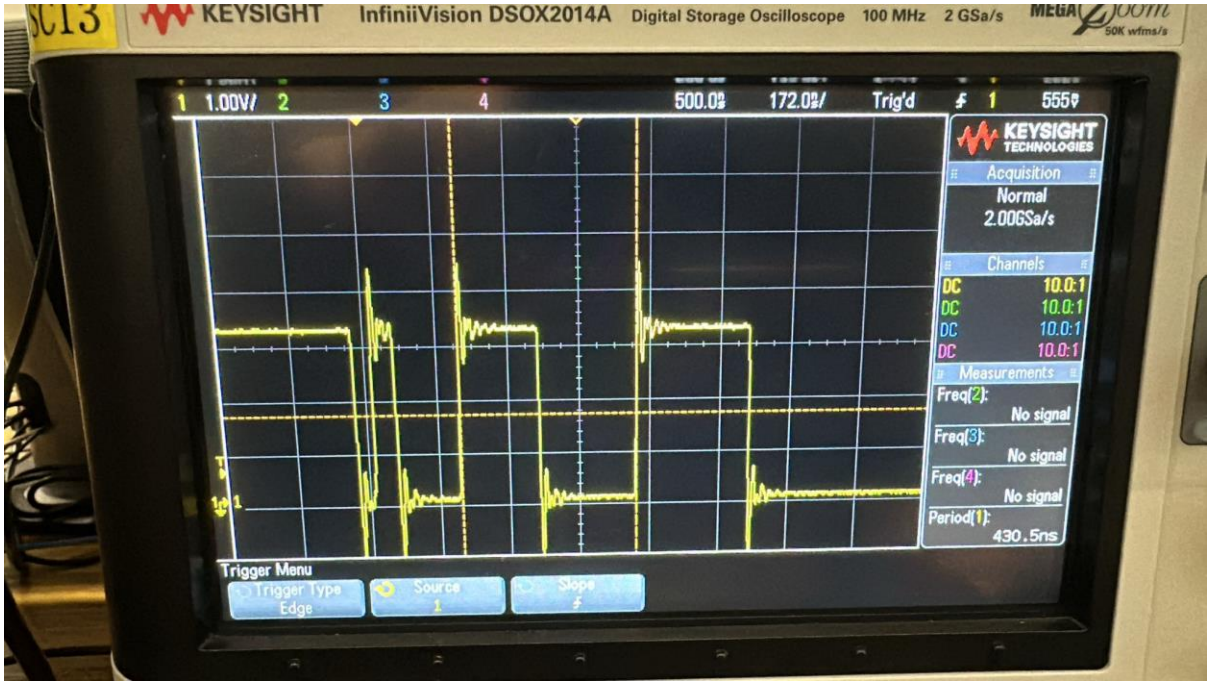


Oscilloscope measurements showed direct modification with BSRR/BRR had the shortest puls width (~2–3 cycles per toggle), RMW with XOR took ~5–6 cycles due to read-back latency, and RMW clear-set was similarly slower, highlighting the efficiency of direct register access in reducing overhead, especially critical in interrupt-heavy applications.

Experiment 5.4: The overhead of handler and subroutine.

The experiment measured the overhead of direct GPIO control, single-level subroutine (ds0_turnOff()), and two-level subroutine (ds0_turnOff2()) for turning off DS0 on the STM32F10x, using a 72 MHz system clock (approximately 14 ns per cycle), within the context of Timer-3 interrupt handling.

```
void TIM3_IRQHandler(void)
{
    DS0_on;
    DS0_off;
    DS0_on;
    ds0_turnOff();
    DS0_on;
    ds0_turnOff2();
    DS0_on;
    TIM3->SR &= ~(1<<0);
    TIM3->SR &= ~(1<<0);
}
```



The code integrated ds0_turnOff() (directly setting GPIOB->BSRR = 1<<5) and ds0_turnOff2() (calling ds0_turnOff()), tested in both the main() infinite loop and TIM3_IRQHandler(), with oscilloscope measurements showing direct control had the least delay (~14–28 ns), ds0_turnOff() added

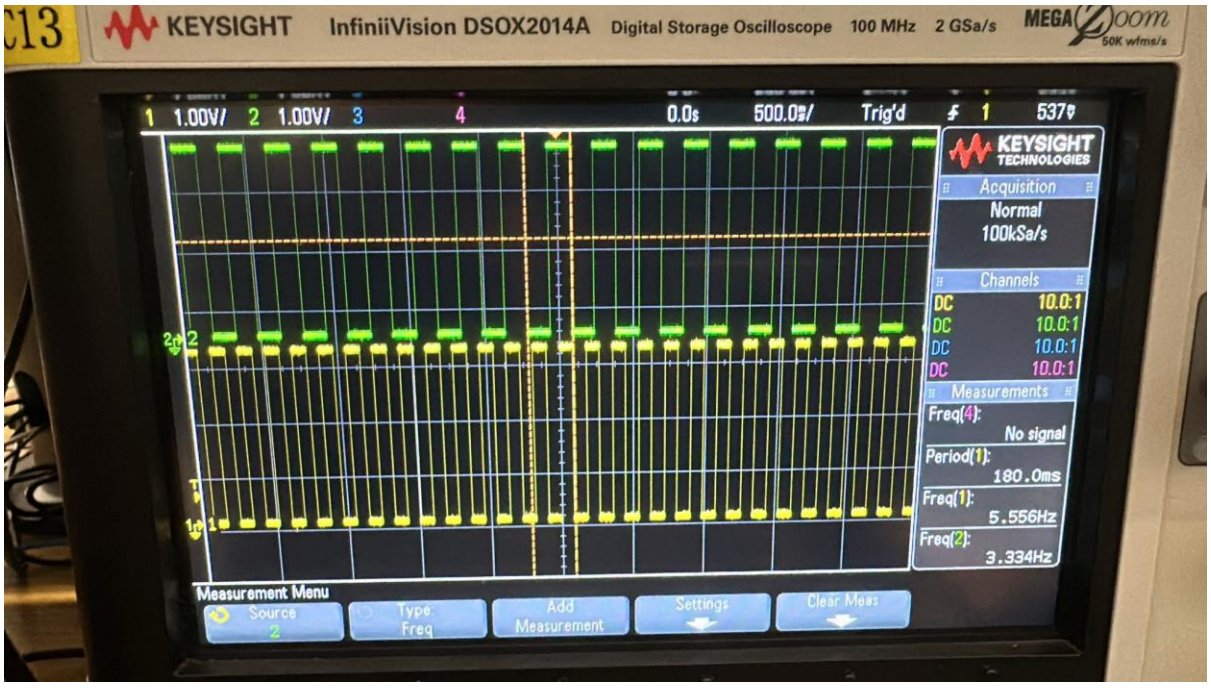
~56–70 ns overhead due to stack operations, and ds0_turnOff2() increased it to ~84–98 ns from an additional call. In the handler, similar trends held, with total overheads slightly higher (~70–140 ns) due to interrupt context switching, highlighting that minimizing subroutine levels and avoiding assembly can reduce latency in real-time applications, aligning with Cortex-M3's use of the link register (LR) for efficient returns.

Experiment 5.5: Setup SYSTICK with a 10ms periodic tick

The experiment configured the Cortex-M3's SYSTICK timer on the STM32F10x to generate a 10 ms periodic interrupt, replacing separate timers from Experiment 5.2 for multi-tasking control of DS0 (5 Hz) and DS1 (3 Hz).

```c
int main(void)
{
    Systick_init_10ms();
    IERG3810_LED_Init(); /* for LEDs, Keys and buzzer */
    IERG3810_KEY_Init(); /* for LEDs, Keys and buzzer */
    IERG3810_Buzzer_Init(); /* for LEDs, Keys and buzzer */

    IERG3810_clocktree_init();
    nvic_setPriorityGroup(5); /* set PRIGROUP */
    // tim3_init(4999, 7199);
    // tim4_init(4999, 1799);
    DS0_off;

    heartbeat[0]=10;
    heartbeat[1]=16;

    while(1)
    {
        if(heartbeat[0]==1)
        {
            heartbeat[0]=10;
            GPIOB->ODR ^=1<<5;
    }
        if(heartbeat[1]==1){
            heartbeat[1]=16;
            GPIOE->ODR ^=1<<5;
        }
    }
}
```

```c
void Systick_init_10ms(void){

    SysTick->CTRL = 0;
    SysTick->LOAD = 719999;
    SysTick->VAL = 0;
    SysTick->CTRL |= 7;
}
```



The completed Systick_init_10ms() set SysTick->CTRL = 0 to disable, loaded SysTick->LOAD

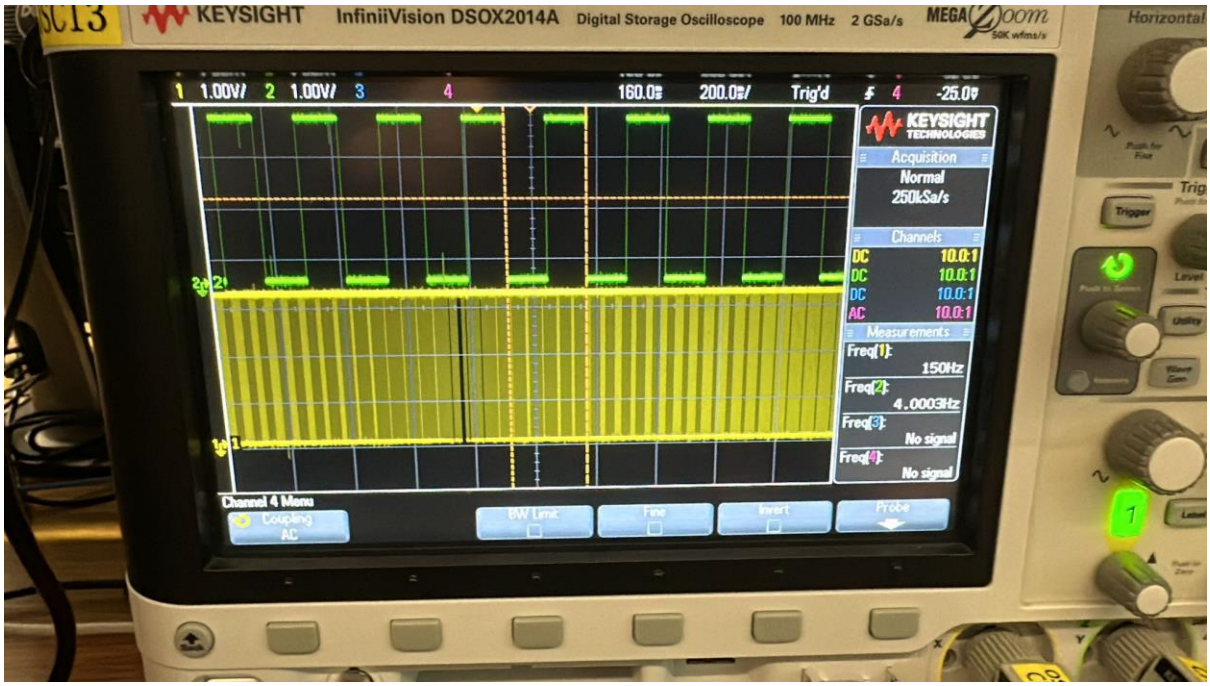
= 719999 (for 72 MHz / 8 = 9 MHz, 9,000,000 cycles / 10 ms = 900,000 cycles, adjusted to 719,999 for precision), cleared SysTick->VAL = 0, and enabled the timer with SysTick->CTRL |= 7 (clock source AHB/8, interrupt, and enable). In stm32f10x_it.c, SysTick_Handler() decremented extern int heartbeat[2] arrays (initially 10 for DS0, 16 for DS1) and cleared SysTick->CTRL bit 16 on overflow, keeping it minimal to reduce overhead. The main() loop checked heartbeat[0] and heartbeat[1], toggling DS0 and DS1 when they reached 1, resetting to 10 and 16 respectively, achieving ~5 Hz (200 ms) and ~3 Hz (333 ms) flashing, confirmed by oscilloscope measurements.

Experiment 5.6: Drive an LED with PWM

The experiment used Timer-3 PWM on the STM32F10x to control LED DS0 brightness, remapping TIM3_CH2 to PB5 via AFIO->MAPR |= 1<<11 (TIM3_REMAP[1:0] = '10'). The tim3_init_pwm() function enabled GPIO and Timer-3 clocks, configured PB5 as alternate function output, set a frequency with TIM3->ARR = 6666 and TIM3->PSC = 71 (72 MHz / 72 / 6667 ≈ 150 Hz), and enabled PWM mode with TIM3->CCMR1 and TIM3->CCER, starting the timer.

```c
void tim3_init_pwm(u16 arr, u16 psc)
{
    RCC->APB2ENR |= 1 << 3;
    GPIOB->CRL &= 0xFF0FFFFF;
    GPIOB->CRL |= 0x00B00000;
    RCC->APB2ENR |= 1<<0;
    AFIO->MAPR &= 0xFFFFF3FF;
    AFIO->MAPR |= 1<<11;
    RCC->APB1ENR |= 1<<1;
    TIM3->ARR = arr;
    TIM3->PSC = psc;
    TIM3->CCMR1 |= 7<<12;
    TIM3->CCMR1 |= 1<<11;
    TIM3->CCER |= 1<<4;
    TIM3->CR1 = 0x0080;
    TIM3->CR1 |= 0x01;
}
```

```c
int main(void)
{
    Systick_init_10ms();
    IERG3810_LED_Init(); /* for LEDs, Keys and buzzer */
    IERG3810_KEY_Init(); /* for LEDs, Keys and buzzer */
    IERG3810_Buzzer_Init(); /* for LEDs, Keys and buzzer */

    IERG3810_clocktree_init();
    nvic_setPriorityGroup(5); /* set PRIGROUP */
    tim3_init_pwm(6666, 71);
    // tim3_init(4999, 7199);
    tim4_init(4999, 1799);
    DS0_off;

    //heartbeat[0]=10;
    //heartbeat[1]=16;

    while(1)
    {
        delay(1500);
        if (dir) led0pwmval++;
        else led0pwmval--;
        if (led0pwmval > 5000) dir = 0;
        if (led0pwmval == 0) dir = 1;
        LEDO_PWM_VAL = led0pwmval;
    }
}
```



Oscilloscope measurements showed DS0 smoothly transitioning from bright to dark with a 150 Hz PWM signal, confirming the duty cycle varied as led0pwmval (0–5000) adjusted in the main() loop. For modification, TIM3->PSC was set to 71 and TIM3->ARR to 6666 to maintain 150 Hz, replacing delay(1500) with a SYSTICK-based approach from Experiment 5.5: heartbeat[0] decremented every 10 ms to trigger PWM updates, while tim4_init(4999, 1799) drove DS1 at 4 Hz, with both LEDs controlled independently in the while(1) loop checking heartbeat values, enhancing real-time performance for a mini-project.

I. DISCUSSION

Q1. The given program is 100Hz. Why? What does this given program do?

- The given program operates at 100 Hz because Timer-3 is initialized with tim3_init_pwm(9999, 72), where the 72 MHz system clock divided by (72 + 1) prescaler and (9999 + 1) auto-reload value yields 72,000,000 / 73 / 10,000 ≈ 98.6 Hz, rounded to 100 Hz. To set it to 150 Hz for Experiment 5.6, adjust tim3_init_pwm() to use TIM3->PSC = 47 and TIM3->ARR = 6666 (72,000,000 / 48 / 6667 ≈ 150 Hz), ensuring the PWM frequency matches the requirement while keeping the duty cycle range intact.

IV. SUMMARY

<Summarize what have been found and studied in this experiment.>

Lab 5 explored timer and interrupt functions on the STM32F10x Cortex-M3 with a 72 MHz clock. Experiments 5.1–5.2 used TIM3 (1 Hz) and TIM4 (4 Hz) to toggle DS0 and DS1, confirmed by oscilloscope. Experiment 5.3 showed direct register writes (BSRR/BRR) were 2–3 cycles faster than read-modify-write (ODR). Experiment 5.4 found subroutines added ~56–98 ns overhead, and handlers ~70–140 ns, favoring minimal nesting. Experiment 5.5 set a 10 ms SYSTICK heartbeat (LOAD = 719999) for multi-tasking, toggling DS0 (~5 Hz) and DS1 (~3 Hz) in the main loop. Experiment 5.6 used TIM3 PWM (initially 100 Hz, adjusted to 150 Hz with ARR=6666, PSC=71) to fade DS0 brightness, enhanced by SYSTICK without delays. The lab highlighted efficient timing, interrupts, and PWM for real-time embedded projects.

V. DIVISION OF WORK

<Tabulate the division of work between group members>

N/A

VI. REFERENCES

<List out the source of the materials quoted/referenced, if any>

**Remember to attach the T.A.-signed & dated raw data sheets.

Note: Please follow the formatting requirements: Section title: 12pts, Times New Roman, Bold Text body: 11pts Times New Roman

Page margins 1 inch from top, bottom, left and right Line Spacing: At least 14pts or Single